

PGR208 Android Programming – Rapport
Kandidatnummer :54

Oppgave 1: Oversikt over funksjonalitet du har laget

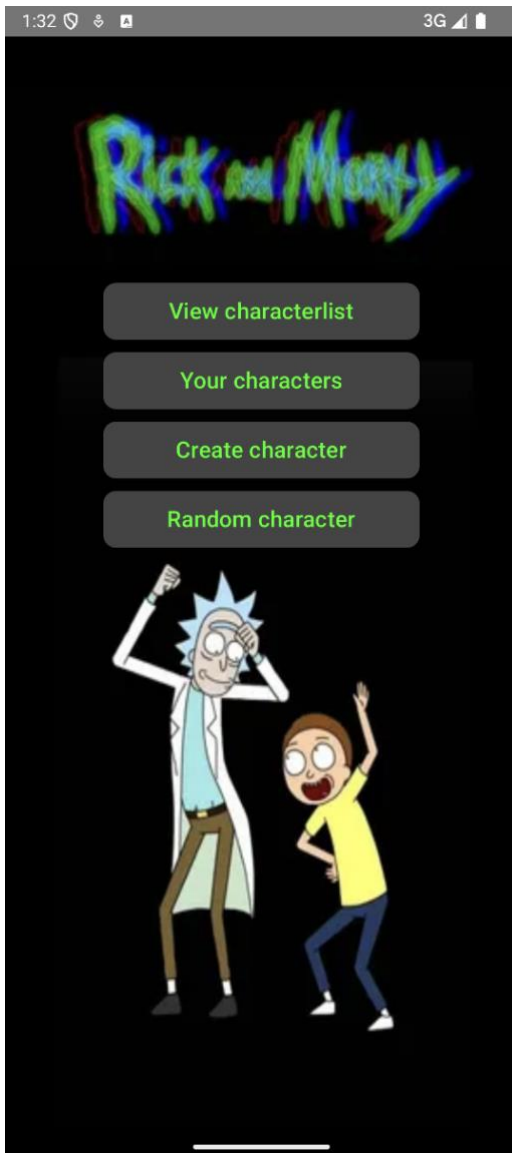
Funksjonalitet	Beskrivelse
LazyColumn Brukt flere steder i oppgaven	Viser det man vil hente frem på en effektiv måte, og med scrolling funksjon tilgjengelig.
Tilbake-knapp: Button (onClick = { navController.popBackStack() })	Lagt inn på alle skjermene for å kunne navigere seg tilbake til «Første siden»
Side #0: FirstPage 1. fun NavigationButton()	1. Reusable knapp for å navigere til de forskjellige sidene.
Side #1: CharacterList 1. fetchCharacters ()	1. Henter listen med karakterer vi har hentet ut fra API-et.
Side #2: UsersCharacters (koden til side #2 og #3 er inspirert av Damke(2023)) 1. fun deleteCharacter () onDeleteClick = { viewModel.deleteCharacter(character.id) }	1. Sletter en karakter i en database, og er knytt til en knapp som kan slette en karakter på UI-en.
Side #3: CreateCharacters 1. fun HeadingShadow()	1. Skyggeeffekt til overskriften 2. Reusable inputfields slik at brukere kan skrive inn data.

2. fun InputField() 3. fun addInventedCharacters () 4. AddButton ()	3. Legger en ny karakter egendefinert karakter i en Room-database. 4. En knapp som legger karakteren til når brukeren har fylt ut alle feltene. Og en If/else-statement for at brukeren er nødt til å fylle ut alle ellers blir de ikke lagt til.
Side #4: RandomCharacter 1. fun fetchRandomCharacter 2. "Random Character"-knappen	1. Henter en tilfeldig karakter fra Character-listen som allerede er hentet ut av API-et, og oppdaterer randomcharacter-state. 2. Når man trykker på knappen utføres fetchRandomCharacter, og en ny karakter blir lastet inn på siden.

Oppgave 2: Skjermdump av samtlige skjermer i appen:

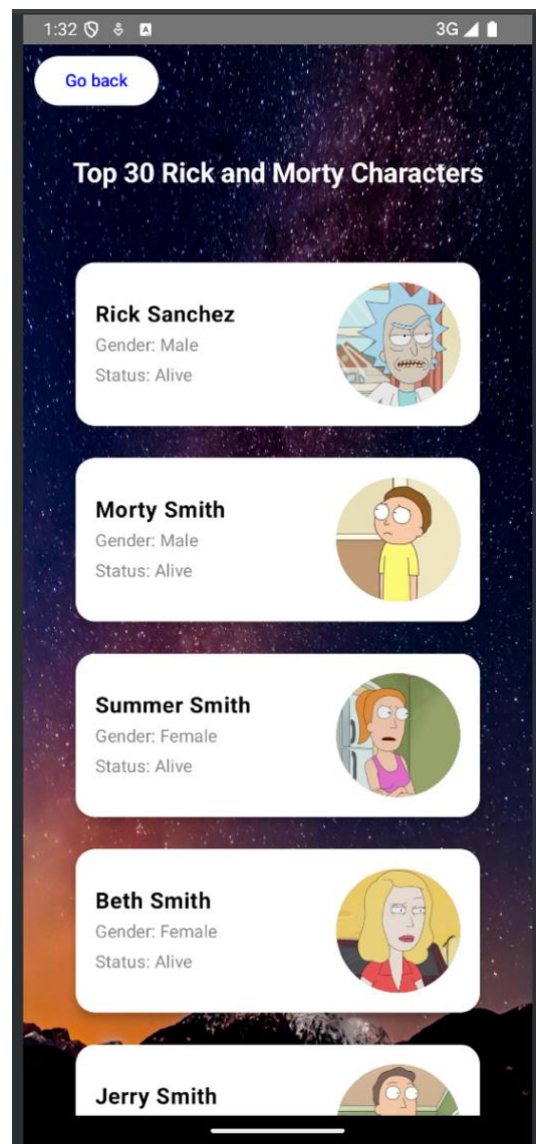
Skjerm #0 (startskjermen)

Forside med navigeringsknapper til alle sidene



Skjerm #1

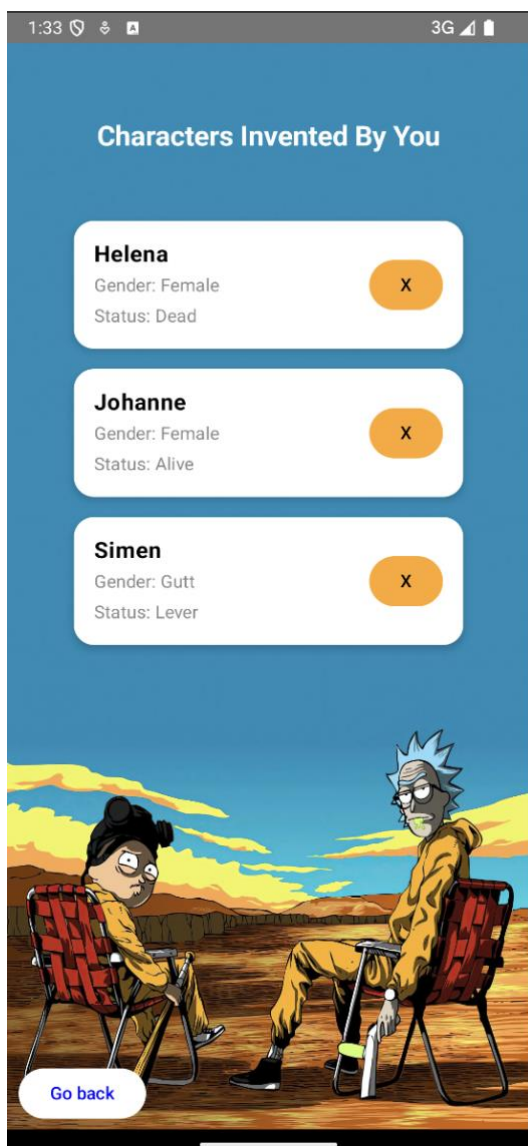
Viser en liste over de 50 første *Rick and Morty*- karakterer fra hentet fra API-et, med bilde, navn, kjønn og status. Brukeren kan bla gjennom listen for å se karakterene, og trykke seg tilbake til startsidene.



Skjerm #2

Skjermen viser en liste over karakterer som brukeren har opprettet. Disse blir lagret i en lokal database, med mulighet til å slette de. Karakterene opprettes av brukeren på skjerm #3.

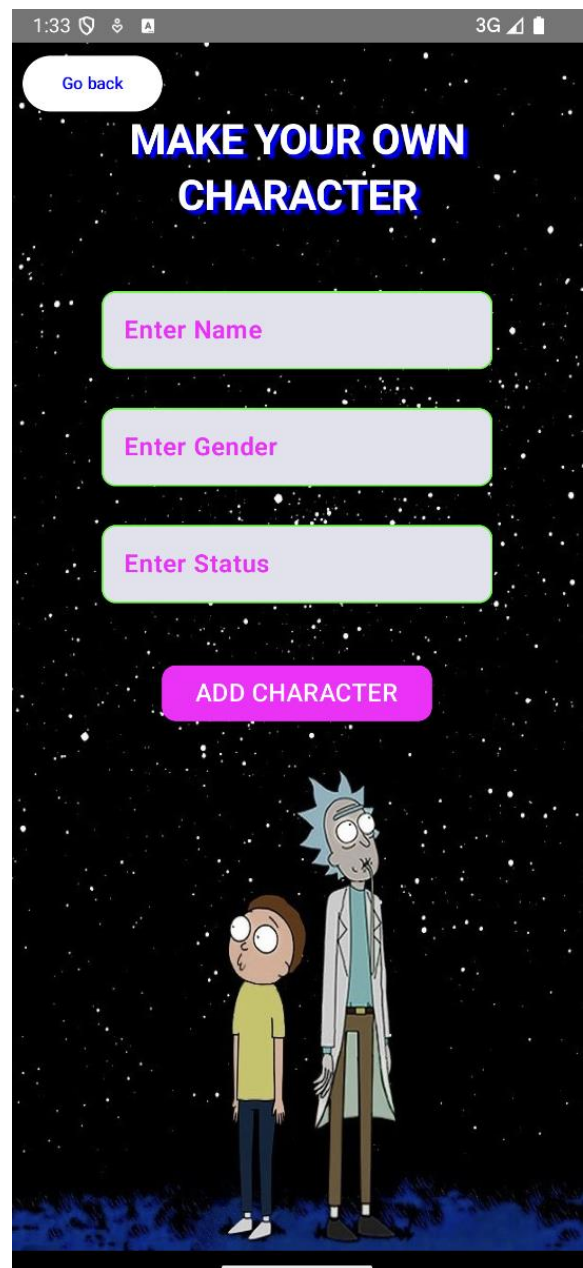
Karakterene vises i samme format som API-karakterene. Brukeren kan også navigere tilbake til startsidene.



Skjerm #3

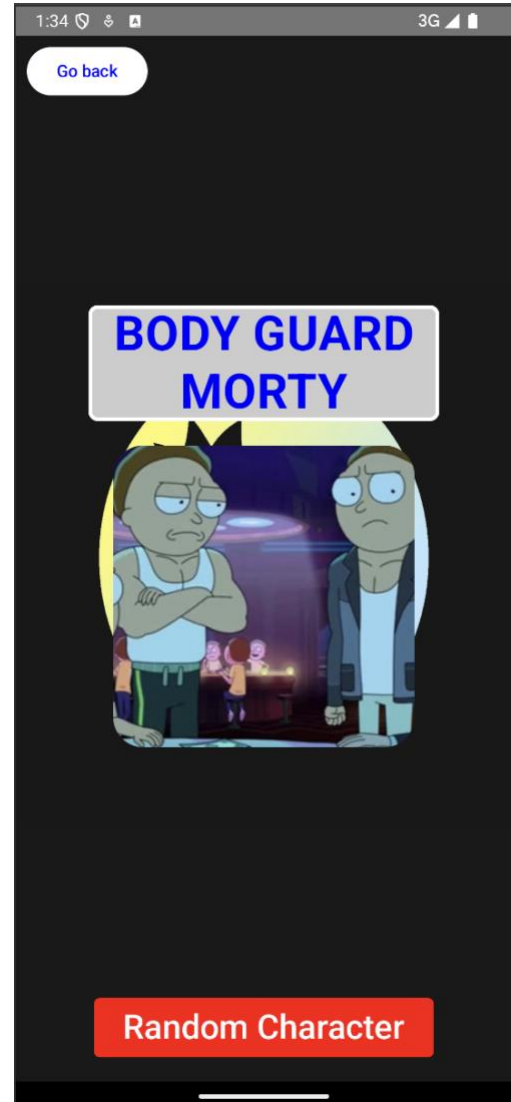
På denne skjermen kan brukeren opprette en egendefinert karakter som blir vist på skjerm #3. Alle felt (navn, kjønn og status) må fylles ut for å fullføre opprettelsen av en karakter. Disse feltene er valgt for å samsvare med egenskapene til de opprinnelige karakterene.

Det er også lagt inn en mulighet til å trykke seg tilbake til start siden.



Skjerm #4

Genererer en tilfeldig karakter fra databasen når brukeren trykker på "Random Character"-knappen. Karakterens navn og bilde vises, og brukeren kan navigere tilbake med "Go back"-knappen.



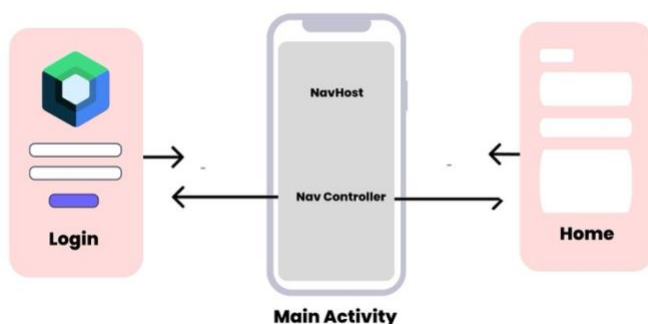
Oppgave 3: Beskrivelser og skjermbilder av hovedteknikker brukt:

Navigation

(NavController og NavHost)

NavController: Verktøy for å styre navigasjonen mellom ulike skjermer i appen.

NavHost: En styrer som knytter en navigasjonsgraf til appens brukergrensesnitt.



Jeg lagde en egen skjerm for å håndtere alle skjermene som oppgaven krevde.

```
@Composable
fun AppNavHost() {

    val navController = rememberNavController()
    val _characterListViewModel: CharacterListViewModel = viewModel()
    val characterList = _characterListViewModel.characters.collectAsState().value

    NavHost(
        navController = navController,
        startDestination = "first_page"
    ) {
        composable(Screen.FirstPage.route) {
            FirstPage(
                onButton1Click = { navController.navigate(Screen.CharacterList.route) },
                onButton2Click = { navController.navigate(Screen.UsersCharacters.route) },
                onButton3Click = { navController.navigate(Screen.CreateCharacters.route) },
                onButton4Click = { navController.navigate(Screen.RandomCharacters.route) }
            )
        }

        composable(Screen.CharacterList.route) {
            CharacterListScreen(
                viewModel = CharacterListViewModel(),
                navController = navController
            )
        }
    }
}
```

1. Først setter jeg opp en navController med rememberNavController() som skal administrere navigasjonen i appen.

2. Deretter setter jeg opp en NavHost der jeg definerer navigasjonsgrafene (alle skjermene til appen og deres rute).

3. For å navigere seg mellom sidene har hver knapp navController.navigate(), og siden jeg vil de skal bli fraktet til.

Instansieres for å klare å åpne den #4. siden som er basert litt på siden #1

```
val _characterListViewModel: CharacterListViewModel = viewModel()
val characterList = _characterListViewModel.characters.collectAsState().value
```

(Nøst,2024)

State Management:

Håndterer og oppdaterer tilstanden i en applikasjon, slik at UI automatisk reflekterer endringer i data som brukerinnt, nettverksrespons eller annen dynamisk informasjon.

ViewModel:

Håndterer logikken og tilstanden for en skjerm, eksponerer data til UI-en og bevarer tilstanden gjennom konfigurasjonsendringer, som skjermrotasjon. Det er anbefalt å bruke en ViewModel per skjerm for å opprettholde en tydelig ansvarsfordeling.

```
class CharacterListViewModel : ViewModel() {

    private val _loading = MutableStateFlow( value: false)
    val loading = _loading.asStateFlow()

    private val _characters = MutableStateFlow<List<Character>>(emptyList())
    val characters = _characters.asStateFlow()

    init {
        fetchCharacters()
    }

    fun fetchCharacters() {
        viewModelScope.launch {
            _loading.value = true
            _characters.value = ApiRepository.getFiftyCharacters()
            _loading.value = false
        }
    }
}
```

MutableStateFlow og StateFlow:

Her brukes MutableStateFlow for å holde på og oppdatere en verdi reaktivt. Verdien oppdateres ved hjelp av *.value*.

Når vi konverterer MutableStateFlow til asStateFlow, blir objektet kun lesbart og kan ikke endres utenfor ViewModel. Dette sikrer dataen fra andre deler av appen.

LiveData:

Brukes for dynamisk og reaktiv datahåndtering, i tilfellet under observeres data fra databasen og oppdateres i UI-en automatisk når dataen endrer seg.

LiveData er spesielt relevant i dette tilfellet fordi dataene på både side 2 og side 3 må være interaktive og oppdatere seg i sanntid for å forbedre brukeropplevelsen.

```
@Query("SELECT * FROM UsersCharacters")
fun getAllCharacters(): LiveData<List<UsersCharacters>>
```

```
val allUsersCharacters: LiveData<List<UsersCharacters>> = usersCharactersDao.getAllCharacters()
```

```
val allUsersCharacters by viewModel.allUsersCharacters.observeAsState(emptyList())
```

Networking

(Koden min for å håndtere networking er en blanding av kodeløsningen fra forelesning 8, og diverse hentet fra søking på google.)

Retrofit:

Retrofit er et HTTP- klientbibliotek for Android som forenkler nettverksforespørsler. Den håndterer HTTP-forespørselen ved å konvertere JSON-svarene fra serveren direkte til Kotlin-objekter for å effektivisere og gjøre det mer lesbart i koden. Denne teknikken brukes for å hente ut, sende eller oppdatere data fra en ekstern server som REST API-er.

Dependencies lagt inn i build.gradle.kts:

```
implementation ("com.squareup.retrofit2:retrofit:2.11.0")
implementation ("com.squareup.retrofit2:converter-gson:2.11.0")
```

User permission lagt inn i Manifest:

```
<uses-permission android:name="android.permission.INTERNET" />
```

Lager klassen og velger infoen vi vil hente ut fra API-et

```
@Entity |
data class Character(

    @PrimaryKey
    val id: Int,
    val name: String,
    val status: String,
    val gender: String,
    val image: String

)
```

Lager en singleton instance for å få «tak i» API-et i et object `ApiRepository`(her skjer håndteringen av dataen vi innhenter) :

```
private val _retrofit =  
    Retrofit.Builder()  
        .client(_httpClient)  
        .baseUrl(BASE_URL)  
        .addConverterFactory(GsonConverterFactory.create())  
        .build()
```

```
private const val BASE_URL = "https://rickandmortyapi.com/api/"
```

Så lager man et interface som brukes som en service for å hente ut dataen fra delen av API-et man vil bruke:

```
interface ApiService {  
  
    @GET("character/{id}")  
    suspend fun getCharactersById(  
        @Path("id") id: String  
    ): Response<List<Character>>  
}
```

I mitt tilfelle skal jeg ha ut karakterene fra API-ET.

Og jeg vil hente de ut via Id, og plassere de i en liste som heter «Character».

Deretter lager man en service objekt ved bruk av Retrofit:

```
private val _apiService = _retrofit.create(ApiService::class.java)
```

Så kaller man på service objektet og funksjonen i `ApiService` for å hente dataen fra API-et:

```
suspend fun getFiftyCharacters(): List<Character> {  
    try {  
        val ids = (1..50).joinToString(separator = ",")  
  
        val response = _apiService.getCharactersById(ids)  
  
        if (response.isSuccessful) {  
            val characters = response.body() ?: emptyList()  
            _characterDao.insertCharacters(characters)  
            return _characterDao.getCharacters()  
        } else {  
            throw Exception("Response var ikke en suksess")  
        }  
    } catch (e: Exception) {  
        Log.e("ApiRepository", "Kunne ikke hente karakterene", e)  
        return _characterDao.getCharacters()  
    }  
}
```

Her lagde jeg en funksjon som henter ut de 50 første karakteren basert på deres Id. Så har jeg lagt inn en if/else- betingelse som sier at hvis API-et ikke går igjennom så får brukeren en feilmelding og eksisterende data returneres fra databasen.

OkHttpClient:

OkHttpClient er, i likhet med Retrofit, en del av HTTP – klientbiblioteket som gjør det enklere å jobbe med REST API-er. Den er med på overvåking og logging av HTTP-forespørsler og -svar. Det er ikke nødvendig å ha denne med, men anbefales for bedre sikkerhet når det kommer til innhenting av data på nettet.

Startet med å legge inn dependency:

```
implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
```

Lager en OkHttpClient-instanse for å kunne bruke den videre i retrofit etterpå :

```
private val _httpClient =
    OkHttpClient.Builder()
        .addInterceptor(
            HttpLoggingInterceptor()
                .setLevel(HttpLoggingInterceptor.Level.BODY)
        )
        .build()
```

Database

Room:

Dette er en teknikk fra ORM (Object-Relational Mapping)- biblioteket for Android som brukes for lagring og henter data fra en lokal SQLite-database, slik at man kan bruke appen/dataen uten internett.

For å bruke denne teknikken må vi igjennom tre hovedkomponenter:

- **Entity:** Opprettelse av en tabell i databasen, der hvert felt er en kolonne, og man må ha en primærnøkkel. Jeg har laget to databaser: én for API-karakterer og én for brukerens karakterer.

```
@Entity
data class Character(

    @PrimaryKey
    val id: Int,
    val name: String,
    val status: String,
    val gender: String,
    val image: String

)
```

```
@Entity(tableName = "UsersCharacters")
data class UsersCharacters(

    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val name: String,
    val gender: String,
    val status: String

)
```

- **DAO (Data Access Object):** Lage et grensesnitt for å få tilgang til dataen i databasen, og utføre forskjellige operasjoner som innsetting, henting, oppdatering og sletting av data.

```
@Dao
interface CharacterDao {

    @Query("SELECT * FROM Character")
    suspend fun getCharacters(): List<Character>

    @Query("SELECT * FROM Character WHERE :id = id")
    suspend fun getCharacterById(id: Int): Character?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCharacters(characters: List<Character>)
```

- **Database:** Lage en abstrakt klasse som utvider RoomDatabase.

```
@Database(
    entities = [Character::class],
    version = 1,
    exportSchema = false
)

abstract class ApiDatabase : RoomDatabase() {
    abstract fun characterDao(): CharacterDao
}
```

For implementasjon i koden må vi starte med dependencies :

```
// Room
val roomVersion = "2.6.1"
implementation ("androidx.room:room-runtime:$roomVersion")
implementation("androidx.room:room-ktx:$roomVersion")
kapt("androidx.room:room-compiler:$roomVersion")
```

Så instansiere et database-objekt. Dette gjør vi i `ApiRepository` siden jeg skal implementere karakterene fra API-et inn i min lokale database:

```
5     private lateinit var _apiDatabase: ApiDatabase
6     private val _characterDao by lazy { _apiDatabase.characterDao() }
7
8     fun initializeDatabase(context: Context) {
9         _apiDatabase = Room.databaseBuilder(
10             context = context,
11             klass = ApiDatabase::class.java,
12             name = "RickAndMortyCharacters"
13         ).build()
14     }
```

Når jeg kaller på karakterene i «`getFiftyCharacters`»- funksjonen passer jeg også på at de blir lastet inn i databasen.

```
        _characterDao.insertCharacters(characters)
        return _characterDao.getCharacters()
```

Oppgave 4: Kvalitet og struktur

For å opprettholde en lett forståelig prosjektstruktur har jeg fokusert på modularitet, tydelig ansvarsfordeling og lesbarhet. Navngiving er gjort konsekvent og beskrivende, og koden prøver å følge etablerte Kotlin-prinsipper.

Kilder:

Damke, M. R. (2023, February 6). How to save data in local Room database in Android Studio. *Medium*. <https://medium.com/@mohitrdamke/how-to-save-data-in-local-room-database-in-android-studio-da0bb332b7ad>

Google. (n.d.). Save data in a local database using Room. *Android Developers*. <https://developer.android.com/training/data-storage/room>

Moradiya, H. (2021, May 25). Getting started with Room database in Android using Kotlin. *Medium*. <https://medium.com/@harimoradiya123/getting-started-with-room-database-in-android-using-kotlin-92f84b6a5e6c>

Nøst, Morten. (2024). *Lecture 4: Navigation & Activity Lifecycle* [PowerPoint-presentasjon]. Høyskolen Kristiania.

Nøst, Morten. (2024). *Lecture 5 App architecture & ViewModel* [PowerPoint-presentasjon]. Høyskolen Kristiania.

Nøst, Morten. (2024). *Lecture 8: HTTP & Retrofit* [PowerPoint-presentasjon]. Høyskolen Kristiania.

Nøst, Morten. (2024). *Lecture 10: Local Storage* [PowerPoint-presentasjon]. Høyskolen Kristiania.